Spring Boot 3.2 + Java 17

Multi-Country Data Architecture

India Region Design — Complete HLD & LLD

API Gateway · CDN Strategy · Latency & Throughput · Circuit Breaker · Redis Cache · Kafka · Kubernetes HPA · Prometheus

<100ms	10K RPS	95%+	Multi-DS
Latency Target	**Throughput**	**CDN Hit Ratio**	**DB Strategy**
P99 API Response	Per Region	Cache Target	Routing DataSource

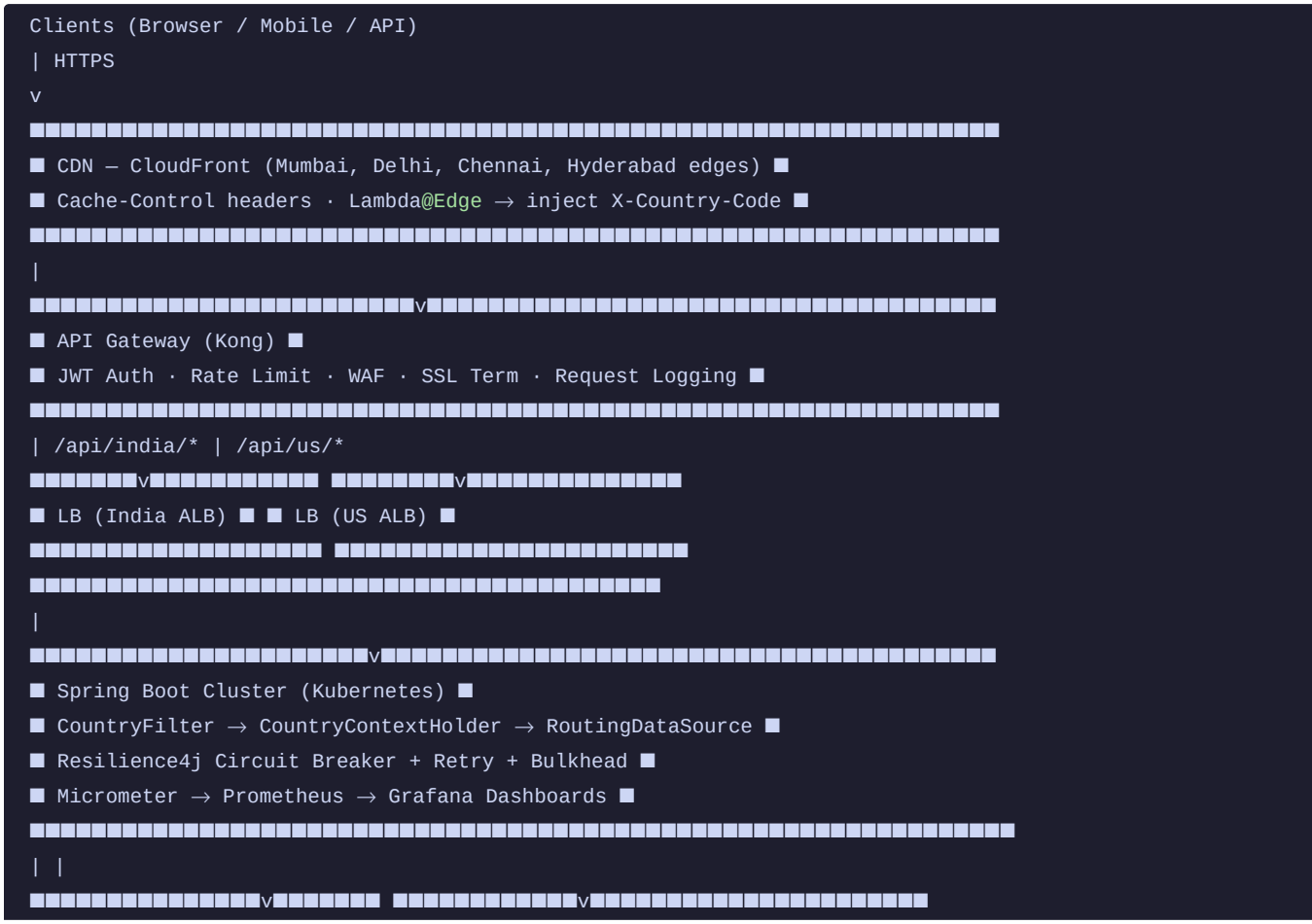
1. HIGH LEVEL DESIGN (HLD)

1.1 System Architecture — Full Stack Flow

All external traffic enters through CloudFront CDN → API Gateway → Load Balancer → Spring Boot Cluster → CountryRoutingDataSource → Country-Specific MySQL RDS. Redis sits as a shared L2 cache. Kafka decouples async writes.

Layer	Component	Technology	Responsibility
1 - Client	Browser / Mobile	React / Flutter	HTTP + X-Country-Code header
2 - CDN	Content Delivery Net.	AWS CloudFront	Edge cache, DDoS, geo-routing
3 - Gateway	API Gateway	Kong / AWS API GW	Auth, rate-limit, WAF, SSL
4 - LB	Load Balancer	Nginx / AWS ALB	Round-robin, health-check
5 - App	Spring Boot Cluster	Java 17 / Boot 3.2	Business logic + routing
6 - Cache	Redis Cluster	Redis 7.x	L2 cache, sessions
7 - Queue	Message Broker	Apache Kafka	Async writes, retry
8 - DB	Country Databases	MySQL 8 RDS	Persistent storage per region

1.2 Architecture Diagram



```
■ Redis Cluster (L2) ■ ■ Kafka (Async write / events) ■
■■■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
| cache miss | consume
■■■■■■■■■■■■■■■■v■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ CountryRoutingDataSource ■
■ "india" → MySQL RDS ap-south-1 (india_db) ■
■ "us" → MySQL RDS us-east-1 (us_db) ■
■ "uk" → MySQL RDS eu-west-2 (uk_db) ■
■ "global" → MySQL RDS us-east-1 (global_db) [default] ■
■■■■■■■■■■■■■■■■■■■
```

```
origins: ["https://india.example.com"]
methods: [GET, POST, PUT, DELETE]
```

2. LOW LEVEL DESIGN — CORE CLASSES

2.1 CountryEnum.java

```java
package com.india.architecture.context;

import lombok.Getter;

// Maps every country to: datasource key, ISO code, AWS region
@Getter
public enum CountryEnum {

INDIA ("india", "IN", "ap-south-1"),
US ("us", "US", "us-east-1"),
UK ("uk", "UK", "eu-west-2"),
GLOBAL("global", "GL", "us-east-1"); // default fallback

private final String dsKey; // HikariCP datasource key
private final String isoCode; // ISO 3166-1 alpha-2
private final String awsRegion; // AWS deployment region

CountryEnum(String dsKey, String isoCode, String awsRegion) {
this.dsKey = dsKey;
this.isoCode = isoCode;
this.awsRegion = awsRegion;
}

// Accepts both "india" and "IN" — case insensitive
public static CountryEnum fromHeader(String value) {
if (value == null || value.isBlank()) return GLOBAL;
for (CountryEnum c : values()) {
if (c.dsKey.equalsIgnoreCase(value) ||
c.isoCode.equalsIgnoreCase(value)) {
return c;
}
}
return GLOBAL;
}
}
```

2.2 CountryContextHolder.java

```java
package com.india.architecture.context;

// Thread-local country context — one slot per request thread
// Lifecycle: CountryFilter.set() → Service reads → Filter.clear()
public final class CountryContextHolder {

private static final ThreadLocal<CountryEnum> CONTEXT =
ThreadLocal.withInitial(() -> CountryEnum.GLOBAL);

private CountryContextHolder() {} // prevent instantiation

public static void set(CountryEnum country) {
CONTEXT.set(country);
}

public static CountryEnum get() {
return CONTEXT.get();
}
```

```
// Returns "india" | "us" | "uk" | "global"
public static String dsKey() {
return CONTEXT.get().getDsKey();
}
// Returns "IN" | "US" | "UK" | "GL"
public static String isoCode() {
return CONTEXT.get().getIsoCode();
}
// CRITICAL: call in finally block — prevents ThreadLocal memory leak
public static void clear() {
CONTEXT.remove();
}
}
```

2.3 CountryRoutingDataSource.java

```
package com.india.architecture.routing;
import com.india.architecture.context.CountryContextHolder;
import lombok.extern.slf4j.Slf4j;
import org.springframework.jdbc.datasource.lookup.AbstractRoutingDataSource;
import javax.sql.DataSource;
// Spring calls determineCurrentLookupKey() before EVERY JDBC operation
@Slf4j
public class CountryRoutingDataSource extends AbstractRoutingDataSource {
@Override
protected Object determineCurrentLookupKey() {
String key = CountryContextHolder.dsKey();
log.debug("[DS-ROUTING] Routing JDBC → datasource key: '{}'", key);
return key; // matches keys in DataSourceConfig targets map
}
@Override
protected DataSource resolveSpecifiedDataSource(Object dataSource) {
if (dataSource == null) {
log.warn("[DS-ROUTING] Key not found, using GLOBAL fallback");
return getResolvedDefaultDataSource();
}
return super.resolveSpecifiedDataSource(dataSource);
}
}
```

2.4 DataSourceConfig.java

```
package com.india.architecture.config;
import com.india.architecture.context.CountryEnum;
import com.india.architecture.routing.CountryRoutingDataSource;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.boot.jdbc.DataSourceBuilder;
import org.springframework.context.annotation.*;
import javax.sql.DataSource;
import java.util.*;
```

```java
@Configuration
public class DataSourceConfig {
// Each bean reads from datasources.india.* in application.yml
@Bean("indiaDS")
@ConfigurationProperties(prefix = "datasources.india")
public DataSource indiaDataSource() {
return DataSourceBuilder.create().build();
}
@Bean("usDS")
@ConfigurationProperties(prefix = "datasources.us")
public DataSource usDataSource() {
return DataSourceBuilder.create().build();
}
@Bean("ukDS")
@ConfigurationProperties(prefix = "datasources.uk")
public DataSource ukDataSource() {
return DataSourceBuilder.create().build();
}
@Bean("globalDS")
@ConfigurationProperties(prefix = "datasources.global")
public DataSource globalDataSource() {
return DataSourceBuilder.create().build();
}
@Bean("routingDataSource")
@Primary
public DataSource routingDataSource(
@Qualifier("indiaDS") DataSource india,
@Qualifier("usDS") DataSource us,
@Qualifier("ukDS") DataSource uk,
@Qualifier("globalDS") DataSource global) {
Map<Object, Object> targets = new HashMap<>();
targets.put(CountryEnum.INDIA.getDsKey(), india); // "india"
targets.put(CountryEnum.US.getDsKey(), us); // "us"
targets.put(CountryEnum.UK.getDsKey(), uk); // "uk"
targets.put(CountryEnum.GLOBAL.getDsKey(), global); // "global"
CountryRoutingDataSource routing = new CountryRoutingDataSource();
routing.setTargetDataSources(targets);
routing.setDefaultTargetDataSource(global); // fallback
routing.afterPropertiesSet();
return routing;
}
}
```

3. LLD — FILTER, ENTITY & SERVICE

3.1 CountryFilter.java (Order = 1)

```java
package com.india.architecture.filter;
import com.india.architecture.context.*;
import jakarta.servlet.*;
import jakarta.servlet.http.HttpServletRequest;
import lombok.extern.slf4j.Slf4j;
import org.slf4j.MDC;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;
import java.io.IOException;
import java.util.UUID;
// Runs FIRST on every HTTP request — sets country context
@Component
@Order(1)
@Slf4j
public class CountryFilter implements Filter {
@Override
public void doFilter(ServletRequest req, ServletResponse res,
FilterChain chain) throws IOException, ServletException {
HttpServletRequest request = (HttpServletRequest) req;
// Priority: Header → Query Param → Default GLOBAL
String raw = request.getHeader("X-Country-Code");
if (raw == null || raw.isBlank()) {
raw = request.getParameter("country");
}
CountryEnum country = CountryEnum.fromHeader(raw);
// Correlation ID for distributed tracing
String requestId = request.getHeader("X-Request-ID");
if (requestId == null) requestId = UUID.randomUUID().toString();
CountryContextHolder.set(country);
MDC.put("country", country.getIsoCode());
MDC.put("requestId", requestId);
try {
chain.doFilter(req, res);
} finally {
CountryContextHolder.clear(); // MUST — prevents ThreadLocal leak
MDC.clear();
}
}
}
```

3.2 BaseEntity.java + Customer.java

```java
// BaseEntity.java — inherited by all country-scoped entities
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
```

```
@Getter @Setter
public abstract class BaseEntity {
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
@Column(name = "country_code", nullable = false, length = 2)
private String countryCode; // "IN", "US", "UK"
@CreatedDate
@Column(name = "created_at", updatable = false)
private LocalDateTime createdAt;
@LastModifiedDate
@Column(name = "updated_at")
private LocalDateTime updatedAt;
@Version
private Long version; // Optimistic locking
}
// Customer.java — India-specific fields (GSTIN, PAN, state)
@Entity
@Table(name = "customers", indexes = {
@Index(name = "idx_country", columnList = "country_code"),
@Index(name = "idx_country_state", columnList = "country_code,state"),
@Index(name = "idx_gstin", columnList = "gstin")
})
@Getter @Setter @Builder @NoArgsConstructor @AllArgsConstructor
public class Customer extends BaseEntity {
@NotBlank
private String name;
@Email @NotBlank
private String email;
private String state; // Indian state: "Maharashtra", "Karnataka" …
@Pattern(regexp = "^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z]{1}[1-9A-Z]{1}Z[0-9A-Z]{1}$")
private String gstin; // GST Identification Number (India only)
@Pattern(regexp = "[A-Z]{5}[0-9]{4}[A-Z]{1}")
private String pan; // Permanent Account Number (India only)
@Enumerated(EnumType.STRING)
private CustomerStatus status = CustomerStatus.ACTIVE;
public enum CustomerStatus { ACTIVE, INACTIVE, SUSPENDED }
}
```

3.3 CountryDataService.java — Cache + Circuit Breaker

```
@Service
@RequiredArgsConstructor
@Slf4j
public class CountryDataService {
private final CustomerRepository customerRepository;
// READ: Cache-Aside (Redis L2) + Circuit Breaker
@Cacheable(value = "customers",
key = "T(com.india.architecture.context.CountryContextHolder)"
+ ".isoCode() + ':' + #state")
```

```
@CircuitBreaker(name = "indiaDB", fallbackMethod = "fallbackByState")
@Retry(name = "indiaDB")
public List<Customer> getCustomersByState(String state) {
String iso = CountryContextHolder.isoCode();
return customerRepository.findByCountryCodeAndState(iso, state);
}
// WRITE: Evict cache + auto-set country from context
@Transactional
@CacheEvict(value = "customers", allEntries = true)
public Customer save(Customer customer) {
customer.setCountryCode(CountryContextHolder.isoCode());
return customerRepository.save(customer);
}
// PAGINATED: always paginate — never load all rows!
public Page<Customer> getPagedCustomers(Pageable pageable) {
return customerRepository.findActiveByCountryCode(
CountryContextHolder.isoCode(), pageable);
}
// CROSS-COUNTRY: switch context per loop iteration
public Map<String, Long> countByAllCountries() {
Map<String, Long> result = new LinkedHashMap<>();
for (CountryEnum c : CountryEnum.values()) {
if (c == CountryEnum.GLOBAL) continue;
CountryContextHolder.set(c);
try {
result.put(c.name(),
customerRepository.countByCountryCode(c.getIsoCode()));
} finally {
CountryContextHolder.clear(); // clear after each country
}
}
return result;
}
// Fallback when circuit is OPEN
public List<Customer> fallbackByState(String state, Exception ex) {
log.error("[FALLBACK] India DB unavailable state={}: {}", state, ex.getMessage());
return List.of(); // graceful empty response
}
}
```

4. LLD — STARTUP LOADER & REST API

4.1 IndiaDataLoader.java (ApplicationRunner)

```java
@Component
@RequiredArgsConstructor
@Slf4j
public class IndiaDataLoader implements ApplicationRunner {
private final CustomerRepository customerRepository;
private final CacheManager cacheManager;
// Runs ONCE after Spring Boot startup
// Pre-warms Redis cache with India reference data
@Override
public void run(ApplicationArguments args) {
log.info("■■■
```

```
Page<Customer> result = service.getPagedCustomers(
PageRequest.of(page, size, Sort.by("createdAt").descending()));
return ResponseEntity.ok(result);
}
// GET /api/v1/customers/by-state?state=Maharashtra
@GetMapping("/by-state")
public ResponseEntity<List<Customer>> byState(@RequestParam String state) {
return ResponseEntity.ok(service.getCustomersByState(state));
}
// POST /api/v1/customers
@PostMapping
public ResponseEntity<Customer> create(@Valid @RequestBody Customer customer) {
return ResponseEntity.status(201).body(service.save(customer));
}
// PUT /api/v1/customers/{id}
@PutMapping("/{id}")
public ResponseEntity<Customer> update(
@PathVariable Long id,
@Valid @RequestBody Customer patch) {
return ResponseEntity.ok(service.update(id, patch));
}
// DELETE /api/v1/customers/{id}
@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void delete(@PathVariable Long id) {
service.delete(id);
}
// GET /api/v1/customers/stats — count per country
@GetMapping("/stats")
public ResponseEntity<Map<String, Long>> stats() {
return ResponseEntity.ok(service.countByAllCountries());
}
}
```

5. LATENCY & THROUGHPUT MANAGEMENT

5.1 Latency Sources & Targets

Source	Typical Delay	Strategy	Target
India↔US cross-region	150-300ms	India-local RDS ap-south-1	<5ms
SSL Handshake	30-100ms	SSL termination at CDN	0ms
Connection pool wait	10-500ms	HikariCP min-idle=5	<3ms
Cache miss (DB query)	50-500ms	Redis L2 + DB indexes	<10ms
JPA N+1 query	100-2000ms	JOIN FETCH + @BatchSize	<20ms
GC pause	10-200ms	G1GC MaxGCPauseMillis=100	<20ms

5.2 HikariCP + Redis Cache Config (application.yml)

```
datasources:
india:
hikari:
pool-name: IndiaPool
maximum-pool-size: 20 # = (CPU cores * 2) + disk spindles
minimum-idle: 5 # 5 warm connections always ready
connection-timeout: 3000 # Fail fast: 3s max wait
idle-timeout: 600000 # 10 min before close
max-lifetime: 1800000 # 30 min recycle (before MySQL timeout)
keepalive-time: 60000 # Ping every 60s vs firewall drop
spring:
data:
redis:
timeout: 2000ms
lettuce:
pool:
max-active: 20
min-idle: 5
```

5.3 Redis Cache-Aside Pattern (CacheConfig.java)

```
@Configuration
@EnableCaching
public class CacheConfig {
@Bean @Primary
public CacheManager redisCacheManager(RedisConnectionFactory factory) {
RedisCacheConfiguration base = RedisCacheConfiguration.defaultCacheConfig()
.entryTtl(Duration.ofMinutes(30))
.disableCachingNullValues();
Map<String, RedisCacheConfiguration> perCache = new HashMap<>();
perCache.put("customers", base.entryTtl(Duration.ofMinutes(60)));
perCache.put("products", base.entryTtl(Duration.ofMinutes(30)));
```

```
perCache.put("masterData", base.entryTtl(Duration.ofHours(24)));
return RedisCacheManager.builder(factory)
.cacheDefaults(base)
.withInitialCacheConfigurations(perCache)
.build();
}
// L1: JVM Caffeine cache — fastest, per-instance, 5-min TTL
@Bean("caffeineCacheManager")
public CacheManager caffeineCacheManager() {
CaffeineCacheManager mgr = new CaffeineCacheManager();
mgr.setCaffeineSpec(CaffeineSpec.parse(
"maximumSize=10000,expireAfterWrite=5m"));
return mgr;
}
}
```

5.4 Resilience4j Circuit Breaker + Retry

```
resilience4j:
circuitbreaker:
instances:
indiaDB:
sliding-window-size: 20 # Evaluate last 20 calls
failure-rate-threshold: 50 # Open if 50% fail
wait-duration-in-open-state: 30s # Wait before half-open
permitted-calls-in-half-open-state: 5
retry:
instances:
indiaDB:
max-attempts: 3
wait-duration: 200ms
exponential-backoff-multiplier: 2.0 # 200 → 400 → 800ms
retry-exceptions:
- java.sql.SQLException
- org.springframework.dao.DataAccessException
```

5.5 Throughput — Horizontal Scaling & Rate Limiting

Load Level	Instances	DB Connections	Target RPS
Baseline	2	2 × 20 = 40	500 RPS
Normal	5	5 × 20 = 100	2,500 RPS
Peak	20	20 × 20 = 400	10,000 RPS
Burst (HPA)	50	Capped at 500	25,000 RPS

```
// RateLimitFilter.java — Token bucket per (clientId + country)
private Bucket createBucket(String isoCode) {
int limit = switch (isoCode) {
case "IN" -> 1000; // India: 1000 req/min
case "US" -> 2000; // US: 2000 req/min
```

```
case "UK" -> 1500; // UK: 1500 req/min
default -> 500;
};
return Bucket.builder()
.addLimit(Bandwidth.classic(limit,
Refill.greedy(limit, Duration.ofMinutes(1))))
.build();
}
// Async Bulkhead — dedicated thread pool per country
@Bean("indiaTaskExecutor")
public Executor indiaTaskExecutor() {
ThreadPoolTaskExecutor exec = new ThreadPoolTaskExecutor();
exec.setCorePoolSize(10);
exec.setMaxPoolSize(50);
exec.setQueueCapacity(200);
exec.setThreadNamePrefix("india-async-");
exec.setRejectedExecutionHandler(new CallerRunsPolicy());
exec.initialize();
return exec;
}
```

6. MONITORING, SECURITY & DEPLOYMENT

6.1 Micrometer Custom Metrics (CountryMetrics.java)

```
@Component
public class CountryMetrics {
private final MeterRegistry registry;
public void recordApiRequest(String isoCode, String status) {
Counter.builder("india.api.requests")
.description("API requests per country")
.tag("country", isoCode)
.tag("status", status) // "success" | "error"
.register(registry)
.increment();
}
public void recordDbQuery(String isoCode, String type, long durationMs) {
Timer.builder("india.db.query.duration")
.tag("country", isoCode)
.tag("query_type", type) // "read" | "write"
.publishPercentiles(0.5, 0.95, 0.99)
.register(registry)
.record(Duration.ofMillis(durationMs));
}
}
// Grafana alerts:
// india.api.requests P99 > 500ms → PagerDuty
// india.db.query.duration > 1s → Slack alert
// Circuit breaker OPEN → Immediate escalation
```

6.2 Monitoring Stack

Tool	Purpose	Key Config
Prometheus	Metrics scraping	Scrape /actuator/prometheus every 15s
Grafana	Dashboards + Alerts	India RPS, P99 latency, circuit state
Jaeger	Distributed tracing	Trace India request end-to-end
ELK Stack	Log aggregation	MDC: country, requestId in every log line
PagerDuty	Incident alerting	Alert if P99 > 500ms or error > 1%

6.3 SecurityConfig.java

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
http
.csrf(AbstractHttpConfigurer::disable)
```

```
.sessionManagement(sm ->
sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
.authorizeHttpRequests(auth -> auth
// Public: health, swagger
.requestMatchers("/actuator/health", "/swagger-ui/**",
"/v3/api-docs/**").permitAll()
// India endpoints: require INDIA_ADMIN role
.requestMatchers("/api/v1/customers/**")
.hasAnyRole("USER","ADMIN","INDIA_ADMIN")
.requestMatchers("/api/v1/customers/stats")
.hasRole("ADMIN")
.anyRequest().authenticated()
)
// Stateless JWT via OAuth2 Resource Server
.oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt -> {}));
return http.build();
}
}
```

6.4 Kubernetes HPA — Auto-scaling

```
# k8s HPA — scales Spring Boot pods based on CPU
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
name: india-service-hpa
spec:
scaleTargetRef:
apiVersion: apps/v1
kind: Deployment
name: india-customer-service
minReplicas: 2
maxReplicas: 50
metrics:
- type: Resource
resource:
name: cpu
target:
type: Utilization
averageUtilization: 60 # Scale at 60% CPU
# JVM flags for container awareness
JAVA_OPTS: >
-server
-XX:+UseG1GC
-XX:MaxGCPauseMillis=100
-XX:+UseContainerSupport
-XX:MaxRAMPercentage=75.0
# Multi-region deployment
# India: EKS ap-south-1 (Mumbai) 3 AZs
# US: EKS us-east-1 (Virginia) 3 AZs
# UK: EKS eu-west-2 (London) 3 AZs
```

6.5 Quick Test — cURL Examples

```
# List India customers (paginated)
curl -H "X-Country-Code: india" \
http://localhost:8080/api/v1/customers?page=0&size=20

# Filter by state
curl -H "X-Country-Code: IN" \
"http://localhost:8080/api/v1/customers/by-state?state=Maharashtra"

# Create India customer
curl -X POST -H "X-Country-Code: india" \
-H "Content-Type: application/json" \
-d '{"name":"Raj Kumar","email":"raj@example.in","state":"Karnataka","gstin":"29AAAAA0000A1Z5"}' \
http://localhost:8080/api/v1/customers

# Stats for ALL countries
curl http://localhost:8080/api/v1/customers/stats

# Prometheus metrics
curl http://localhost:8080/actuator/prometheus | grep india
```

7. COMPLETE PROJECT STRUCTURE & CONFIG SUMMARY

7.1 Project File Structure

```
src/main/java/com/india/architecture/
■■■ IndiaArchitectureApplication.java // @SpringBootApplication
■■■ context/
■ ■■■ CountryEnum.java // Enum: INDIA, US, UK, GLOBAL
■ ■■■ CountryContextHolder.java // ThreadLocal holder
■■■ routing/
■ ■■■ CountryRoutingDataSource.java // AbstractRoutingDataSource
■■■ config/
■ ■■■ DataSourceConfig.java // 4 HikariCP datasources
■ ■■■ JpaConfig.java // EntityManagerFactory
■ ■■■ CacheConfig.java // Redis L2 + Caffeine L1
■ ■■■ AsyncConfig.java // Thread pools per country
■■■ filter/
■ ■■■ CountryFilter.java // Order=1, sets context
■ ■■■ RateLimitFilter.java // Order=2, Bucket4j
■■■ entity/
■ ■■■ BaseEntity.java // id, countryCode, audit, version
■ ■■■ Customer.java // GSTIN, PAN, state (India fields)
■■■ repository/
■ ■■■ CustomerRepository.java // JPA + projections
■■■ service/
■ ■■■ CountryDataService.java // Cache-Aside + Circuit Breaker
■■■ loader/
■ ■■■ IndiaDataLoader.java // ApplicationRunner pre-warm
■■■ controller/
■ ■■■ CustomerController.java // REST CRUD + stats
■■■ metrics/
■ ■■■ CountryMetrics.java // Micrometer custom meters
```

```
■■■ security/
■ ■■■ SecurityConfig.java // OAuth2 JWT stateless
■■■ exception/
■■■ GlobalExceptionHandler.java // RFC 7807 ProblemDetail
```

7.2 Production Config Reference

Setting	Value	Reason
Java Version	Java 17	LTS, records, sealed classes
Spring Boot	3.2.3	Latest stable, native support
JVM GC	G1GC, pause=100ms	Low-latency API responses
DB Pool per instance	max=20, min-idle=5	(CPU*2)+disks formula
Redis TTL customers	60 minutes	High hit ratio, acceptable staleness
Circuit breaker	50% failure / 20 calls	Opens fast, protects DB
Retry backoff	200→400→800ms, 3x	Exponential with jitter
Rate limit India	1000 req/min/client	Protect India DB
HPA CPU threshold	60%	Scale before saturation
CDN TTL API config	3600s	High cache hit, low staleness
Kafka partitions	12 per topic	= max consumer instances

This architecture document + Spring Boot project ZIP provides a production-ready template for multi-country data isolation in India. All classes are copy-paste ready for Java 17 + Spring Boot 3.2.3. Follow the README.md in the ZIP for quick start.